

Gestión de Workflows con Apache Oozie



Ivana Sánchez Pérez



Automatización de Pipeline CON OOZIE

Introducción y Arquitectura de Flujo

El objetivo de esta práctica es determinar qué cliente ha gastado más dinero combinando datos de una base de datos relacional (MySQL) y un archivo de transacciones semiestructurado (CSV).

Este documento detalla el procedimiento técnico seguido para la implementación de un flujo de trabajo automatizado (Workflow) utilizando Apache Oozie.

Nota sobre la topología: de Fork a Lineal

Aunque el diseño original planteaba una estructura de fork (ejecución paralela de Sqoop y Pig) para la opción1, no he tenido más remedio que optar por una estructura lineal por razones de optimización de recursos. La máquina virtual (Cloudera Quickstart) cuenta con recursos limitados de CPU y RAM y lanzar los dos procesos pesados de MapReduce simultáneamente me ha provocado bloqueos en el JobTracker.

El flujo lineal que he implementado ha garantizado que cada etapa reciba el 100% de los recursos disponibles, asegurando el estado SUCCEEDED.

OPCION 1: SIGUIENDO LA GUIA

PASO 1: Preparación del Entorno y Carga de Datos

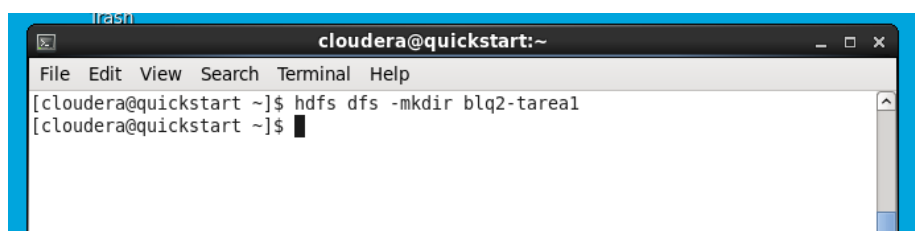
En esta etapa inicial se prepararon los repositorios de datos necesarios para el procesamiento posterior.

Configuración en HDFS

Comenzamos preparando el espacio de trabajo en el sistema de archivos distribuidos (HDFS)

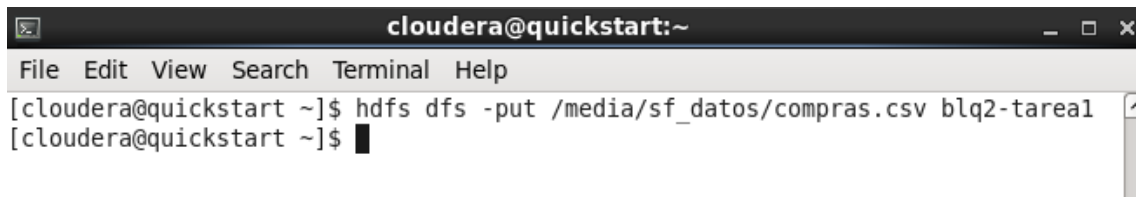
- 1- Se **creó el directorio** de trabajo blq2-tarea1, la raíz del proyecto.

```
hdfs dfs -mkdir blq2-tarea1
```



- 2- Carga del archivo local:** subimos el archivo de transacciones desde la carpeta compartida de la máquina virtual.

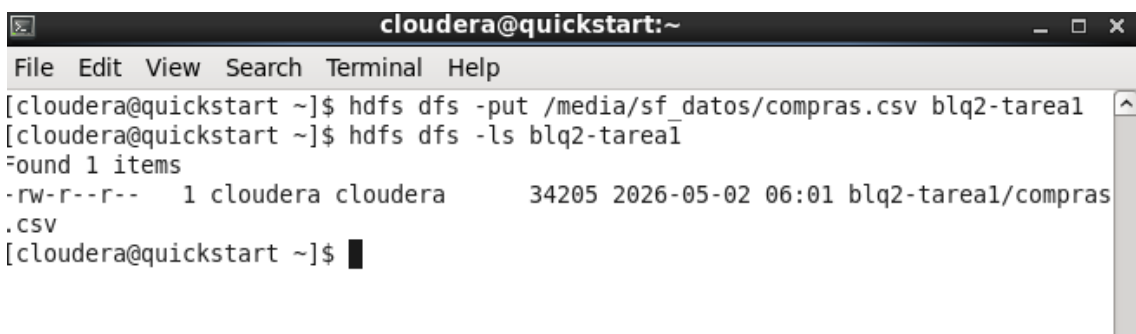
```
hdfs dfs -put ~/media/sf_datos/compras.csv blq2-tarea1
```



```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
[cloudera@quickstart ~]$ hdfs dfs -put /media/sf_datos/compras.csv blq2-tarea1  
[cloudera@quickstart ~]$
```

- 3- Verificación:** Se confirma la existencia del archivo en el clúster.

```
hdfs dfs -ls blq2-tarea1
```



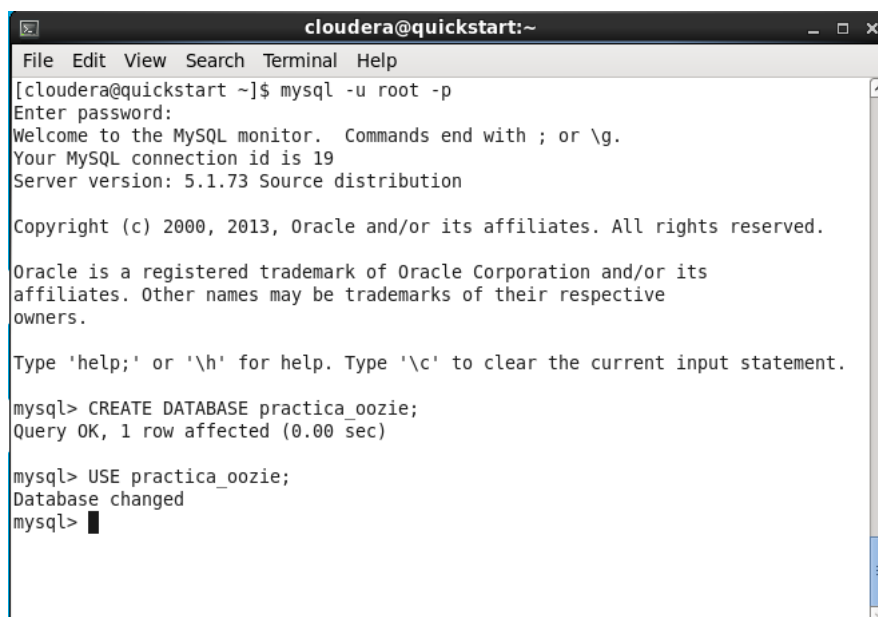
```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
[cloudera@quickstart ~]$ hdfs dfs -put /media/sf_datos/compras.csv blq2-tarea1  
[cloudera@quickstart ~]$ hdfs dfs -ls blq2-tarea1  
Found 1 items  
-rw-r--r--  1 cloudera cloudera      34205 2026-05-02 06:01 blq2-tarea1/compras.csv  
[cloudera@quickstart ~]$
```

Configuración en MySQL

Se prepara la fuente de datos relacional para la ingesta.

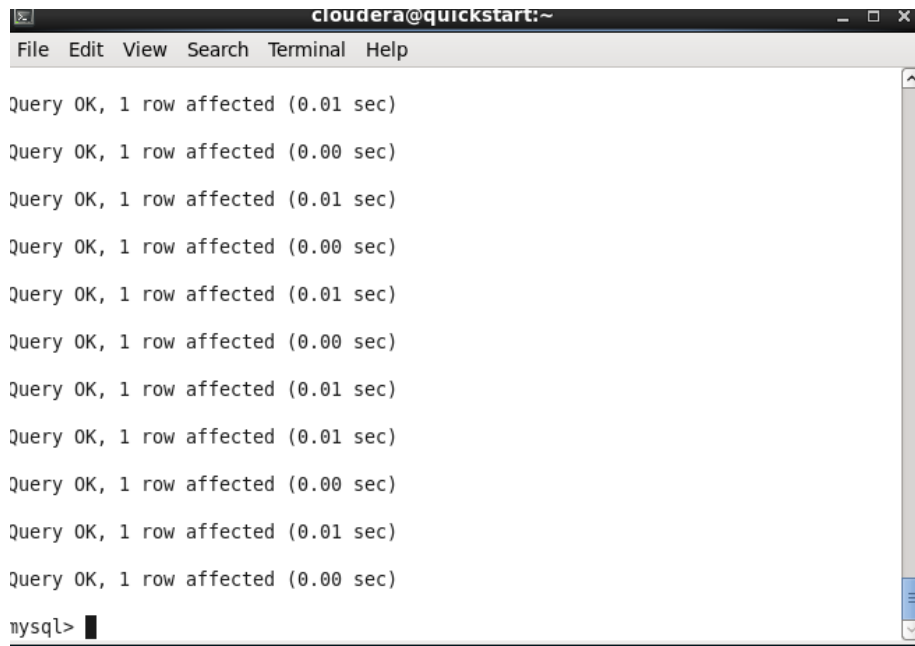
- 1- Acceso y creación:** A través del terminal accedemos a MySQL (mysql -u root -p) y creamos la Base de Datos

```
CREATE DATABASE practica_oozie;  
USE practica_oozie;
```



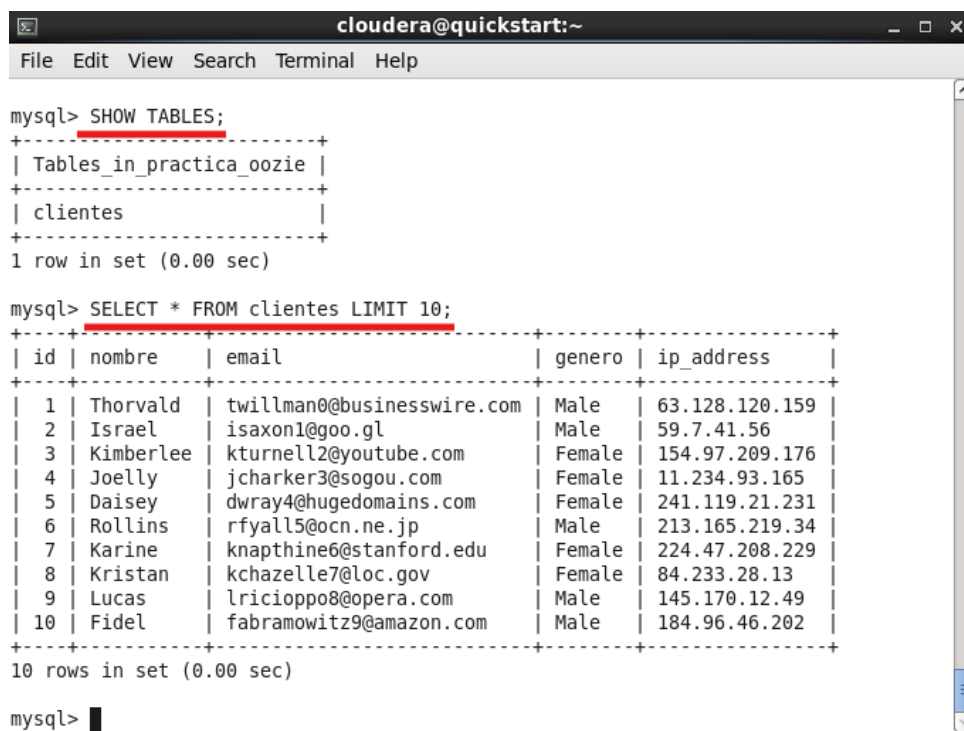
```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
[cloudera@quickstart ~]$ mysql -u root -p  
Enter password:  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 19  
Server version: 5.1.73 Source distribution  
  
Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
mysql> CREATE DATABASE practica_oozie;  
Query OK, 1 row affected (0.00 sec)  
  
mysql> USE practica_oozie;  
Database changed  
mysql>
```

- 2- **Poblamiento:** Se ejecuta el script para crear la tabla de clientes
SOURCE /media/sf_datos/clientes.sql;



```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
  
Query OK, 1 row affected (0.01 sec)  
Query OK, 1 row affected (0.00 sec)  
Query OK, 1 row affected (0.01 sec)  
Query OK, 1 row affected (0.00 sec)  
Query OK, 1 row affected (0.01 sec)  
Query OK, 1 row affected (0.00 sec)  
Query OK, 1 row affected (0.01 sec)  
Query OK, 1 row affected (0.01 sec)  
Query OK, 1 row affected (0.00 sec)  
Query OK, 1 row affected (0.01 sec)  
Query OK, 1 row affected (0.00 sec)  
mysql>
```

- 3- **Validación:** Verificamos que la tabla existe y que tiene registros
SHOW TABLES;
*SELECT * FROM clientes LIMIT 10;*



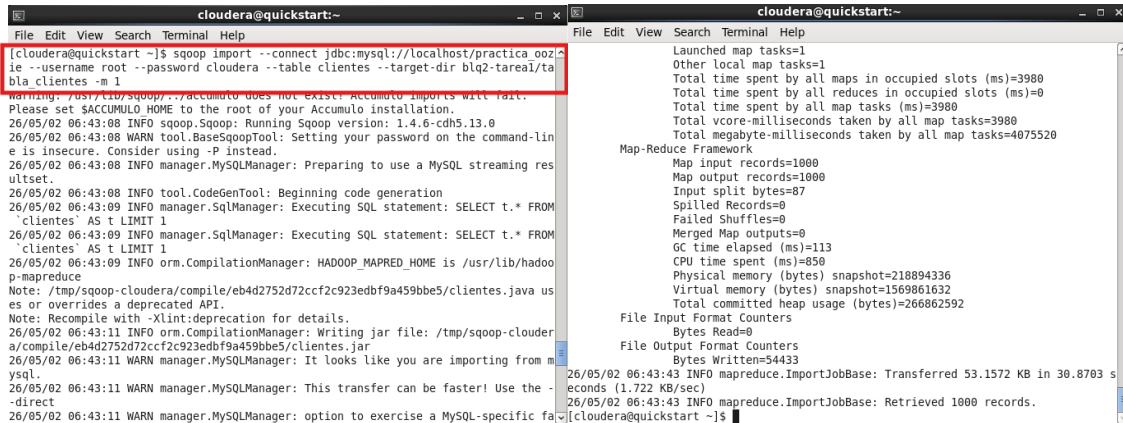
```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
  
mysql> SHOW TABLES;  
+-----+  
| Tables_in_practica_oozie |  
+-----+  
| clientes                  |  
+-----+  
1 row in set (0.00 sec)  
  
mysql> SELECT * FROM clientes LIMIT 10;  
+-----+  
| id | nombre   | email                               | genero | ip_address |  
+-----+  
| 1  | Thorvald | twillman0@businesswire.com         | Male   | 63.128.120.159 |  
| 2  | Israel   | isaxon1@goo.gl                     | Male   | 59.7.41.56      |  
| 3  | Kimberlee | kturnell2@youtube.com              | Female | 154.97.209.176  |  
| 4  | Joelly   | jcharker3@sogou.com                | Female | 11.234.93.165   |  
| 5  | Daisey   | dway4@hugedomains.com              | Female | 241.119.21.231  |  
| 6  | Rollins  | rfyall5@ocn.ne.jp                  | Male   | 213.165.219.34  |  
| 7  | Karine   | knapthine6@stanford.edu             | Female | 224.47.208.229  |  
| 8  | Kristan  | kchazelle7@loc.gov                 | Female | 84.233.28.13    |  
| 9  | Lucas    | lricioppo8@opera.com                | Male   | 145.170.12.49   |  
| 10 | Fidel    | fabramowitz9@amazon.com             | Male   | 184.96.46.202   |  
+-----+  
10 rows in set (0.00 sec)  
  
mysql>
```

PASO 2: Ingesta de Datos con Sqoop

El objetivo es extraer los clientes de MySQL y llevarlos a HDFS para su análisis.

- Comando de importación

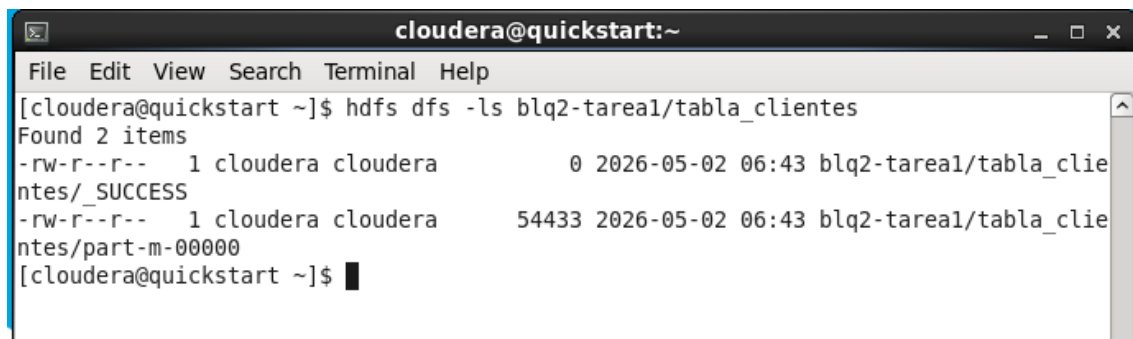
`sqoop import --connect jdbc:mysql://localhost/practica_oozie --username root --password cloudera --table clientes --target-dir blq2-tarea1/tabla_clientes -m 1`



```
cloudera@quickstart:~$ sqoop import --connect jdbc:mysql://localhost/practica_oozie --username root --password cloudera --table clientes --target-dir blq2-tarea1/tabla_clientes -m 1
Warning: /usr/lib/sqoop/../accumulo does not exist: Accumulo imports will fail.
Please set $ACCUMULO_HOME to the root of your Accumulo installation.
26/05/02 06:43:08 INFO sqoop.Sqoop: Running Sqoop version: 1.4.6-cdh5.13.0
26/05/02 06:43:08 WARN tool.BaseSqoopTool: Setting your password on the command-line is insecure. Consider using -P instead.
26/05/02 06:43:08 INFO manager.MySQLManager: Preparing to use a MySQL streaming resultset.
26/05/02 06:43:08 INFO tool.CodeGenTool: Beginning code generation
26/05/02 06:43:09 INFO manager.SqlManager: Executing SQL statement: SELECT t.* FROM `clientes` AS t LIMIT 1
26/05/02 06:43:09 INFO manager.SqlManager: Executing SQL statement: SELECT t.* FROM `clientes` AS t LIMIT 1
26/05/02 06:43:09 INFO orm.CompilationManager: HADOOP_MAPRED_HOME is /usr/lib/hadoop-mapreduce
Note: /tmp/sqoop-cloudera/compile/eb4d2752d72ccf2c923edb9a459bbe5/clientes.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
26/05/02 06:43:11 INFO orm.CompilationManager: Writing jar file: /tmp/sqoop-cloudera/compile/eb4d2752d72ccf2c923edb9a459bbe5/clientes.jar
26/05/02 06:43:11 WARN manager.MySQLManager: It looks like you are importing from mysql.
26/05/02 06:43:11 WARN manager.MySQLManager: This transfer can be faster! Use the --direct
26/05/02 06:43:11 WARN manager.MySQLManager: option to exercise a MySQL-specific fa
Launched map tasks=1
Other local map tasks=1
Total time spent by all maps in occupied slots (ms)=3980
Total time spent by all reduces in occupied slots (ms)=0
Total time spent by all map tasks (ms)=3980
Total vcore-milliseconds taken by all map tasks=3980
Total megabyte-milliseconds taken by all map tasks=4075520
Map-Reduce Framework
  Map input records=1000
  Map output records=1000
  Input split bytes=87
  Spilled Records=0
  Failed Shuffles=0
  Merged Map outputs=0
  GC time elapsed (ms)=113
  CPU time spent (ms)=850
  Physical memory (bytes) snapshot=218894336
  Virtual memory (bytes) snapshot=1569861632
  Total committed heap usage (bytes)=266862592
File Input Format Counters
  Bytes Read=0
File Output Format Counters
  Bytes Written=54433
26/05/02 06:43:43 INFO mapreduce.ImportJobBase: Transferred 53.1572 KB in 30.8703 seconds (1.722 KB/sec)
26/05/02 06:43:43 INFO mapreduce.ImportJobBase: Retrieved 1000 records.
cloudera@quickstart:~$
```

- Resultado: Sqoop genera un archivo en blq2-tarea1/tabla_clientes/part-m-00000 con delimitador de comas

`hdfs dfs -ls blq2-tarea1/tabla_clientes`



```
cloudera@quickstart:~$ hdfs dfs -ls blq2-tarea1/tabla_clientes
Found 2 items
-rw-r--r-- 1 cloudera cloudera 0 2026-05-02 06:43 blq2-tarea1/tabla_clientes/_SUCCESS
-rw-r--r-- 1 cloudera cloudera 54433 2026-05-02 06:43 blq2-tarea1/tabla_clientes/part-m-00000
cloudera@quickstart:~$
```

PASO 3: Procesamiento con Pig

Se realiza la limpieza y transformación de los datos de compras.

- **Lógica del Script (importacion_ventas.pig):**
 - Se cargan los datos usando el delimitador de la fuente.
 - Se filtran los registros con precios nulos.
 - Se almacena el resultado en la carpeta tabla_ventas utilizando punto y coma (;) como separador para evitar conflictos con campos de texto.

```

cloudera@quickstart:~
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ hdfs dfs -ls blq2-tareal/tabla_ventas
Found 2 items
-rw-r--r--  1 cloudera cloudera      0 2026-05-02 06:51 blq2-tareal/tabla_vent
as/_SUCCESS
-rw-r--r--  1 cloudera cloudera 6539 2026-05-02 06:51 blq2-tareal/tabla_vent
as/part-m-00000
[cloudera@quickstart ~]$

```

PASO 4: Análisis Final con Hive

En esta fase se cruzan los datos de clientes y ventas para obtener el resultado de negocio.

- **Configuración de Tablas Externas:** Es crucial definir los delimitadores exactos observados en los archivos (',' para clientes y ';' para ventas).

Entrar en Hive

```

cloudera@quickstart:~
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> USE blq2tarea1;
OK
Time taken: 0.217 seconds
hive>

```

Consulta de Agregación

```

CREATE DATABASE IF NOT EXISTS blq2tarea1;
USE blq2tarea1;

DROP TABLE IF EXISTS ventas;
CREATE EXTERNAL TABLE ventas (
  id INT,
  precio FLOAT
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073'
STORED AS TEXTFILE
LOCATION '/user/cloudera/blq2-tarea1/tabla_ventas';

DROP TABLE IF EXISTS clientes;
CREATE EXTERNAL TABLE clientes (
  id INT,
  nombre STRING,
  email STRING,
  genero STRING,
  ip STRING
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/cloudera/blq2-tarea1/tabla_clientes';

INSERT OVERWRITE DIRECTORY '/user/cloudera/blq2-tarea1/resultado'
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073'
SELECT SUM(v.precio) AS precio_sumado, c.nombre, c.email
FROM ventas v
JOIN clientes c ON (v.id = c.id)
GROUP BY c.id, c.nombre, c.email
ORDER BY precio_sumado DESC
LIMIT 1;

```

```

cloudera@quickstart:~
File Edit View Search Terminal Help
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1777726002872_0009, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1777726002872_0009/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1777726002872_0009
Hadoop job information for Stage-3: number of mappers: 1; number of reducers: 1
2026-05-02 07:45:25,154 Stage-3 map = 0%, reduce = 0%
2026-05-02 07:45:30,842 Stage-3 map = 100%, reduce = 0%, Cumulative CPU 0.63 sec
2026-05-02 07:45:38,445 Stage-3 map = 100%, reduce = 100%, Cumulative CPU 1.34 sec
MapReduce Total cumulative CPU time: 1 seconds 340 msec
Ended Job = job_1777726002872_0009
Moving data to: /user/cloudera/blq2-tarea1/resultado
MapReduce Jobs Launched:
Stage-Stage-2: Map: 1 Reduce: 1 Cumulative CPU: 2.83 sec HDFS Read: 66291 HDFS Write: 96 SUCCESS
Stage-Stage-3: Map: 1 Reduce: 1 Cumulative CPU: 1.34 sec HDFS Read: 4899 HDFS Write: 0 SUCCESS
Total MapReduce CPU Time Spent: 4 seconds 170 msec
OK
Time taken: 73.117 seconds
hive>

```

```

Total MapReduce CPU Time Spent: 3 seconds 650 msec
OK
Time taken: 63.982 seconds
hive> SELECT * FROM ventas LIMIT 5;
OK
4      41.64
34     37.32
31     7.47
6      46.55
46     26.58
Time taken: 0.091 seconds, Fetched: 5 row(s)
hive> SELECT * FROM clientes LIMIT 5;
OK
1      Thorvald      twillman@businesswire.com   Male   63.128.120.159
2      Israel  isaxon1@goo.gl  Male   59.7.41.56
3      Kimberlee  kturnell2@youtube.com  Female 154.97.209.176
4      Joelly  jcharker3@sogou.com    Female 11.234.93.165
5      Daisy  dwwray4@hugedomains.com Female 241.119.21.231
Time taken: 0.063 seconds, Fetched: 5 row(s)
hive>

```


Comprobación resultado en HDFS

Salimos de Hive (exit;) y ejecutamos

```
hdfs dfs -ls blq2-tarea1/resultado
```

```
hdfs dfs -cat blq2-tarea1/resultado/*
```



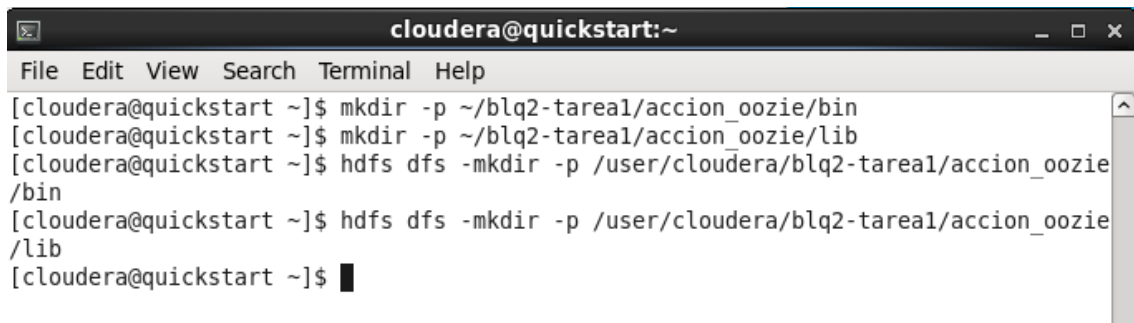
```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
[cloudera@quickstart ~]$ hdfs dfs -ls blq2-tarea1/resultado  
Found 1 items  
-rwxr-xr-x  1 cloudera cloudera      44 2026-05-02 08:03 blq2-tarea1/resultado/  
000000 0  
[cloudera@quickstart ~]$ hdfs dfs -cat blq2-tarea1/resultado/*  
416.3599967956543;Car;cboycott1z@dion.ne.jp  
[cloudera@quickstart ~]$
```

El sistema identifica al cliente con mayor gasto acumulado

PASO 5: Automatización con Oozie

Finalmente se orquestan todos los pasos en un flujo de trabajo automático.

5.1 - Crear estructura de carpetas: Se crean las carpetas bin (para scripts) y lib (para conectores) en HDF

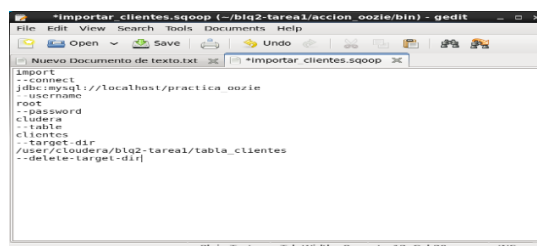


```
cloudera@quickstart:~  
File Edit View Search Terminal Help  
[cloudera@quickstart ~]$ mkdir -p ~/blq2-tarea1/accion_oozie/bin  
[cloudera@quickstart ~]$ mkdir -p ~/blq2-tarea1/accion_oozie/lib  
[cloudera@quickstart ~]$ hdfs dfs -mkdir -p /user/cloudera/blq2-tarea1/accion_oozie  
/bin  
[cloudera@quickstart ~]$ hdfs dfs -mkdir -p /user/cloudera/blq2-tarea1/accion_oozie  
/lib  
[cloudera@quickstart ~]$
```

5.2: Crear los scripts

Script 1: Sqoop (importar clientes desde MySQL)

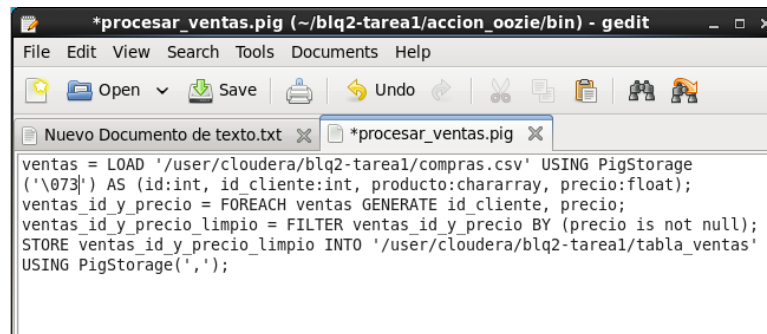
gedit ~/blq2-tarea1/accion_oozie/bin/importar_clientes.sqoop



```
importar_clientes.sqoop (-/blq2-tarea1/accion_oozie/bin) - gedit  
File Edit View Search Tools Documents Help  
Nuevo Documento de texto.txt *importar_clientes.sqoop  
import  
-connect  
jdbc:mysql://localhost/practica_oozie  
-username  
root  
-password  
cloudera  
-table  
clientes  
-target-dir  
/user/cloudera/blq2-tarea1/tabla_clientes  
--delete-target-dir
```


Script 2: Pig (procesar el CSV de ventas)

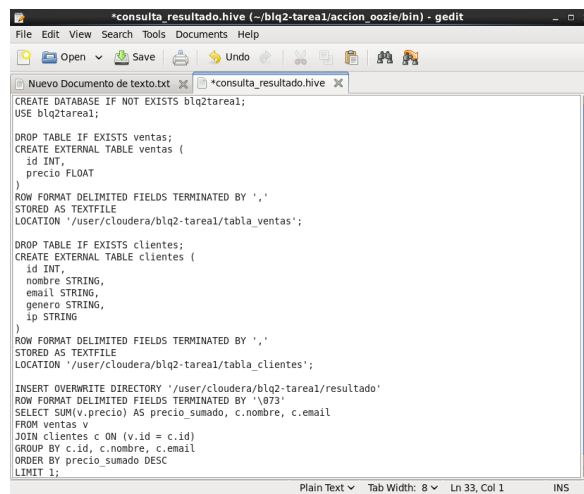
gedit ~/blq2-tarea1/accion_oozie/bin/procesar_ventas.pig



```
*procesar_ventas.pig (~/blq2-tarea1/accion_oozie/bin) - gedit
File Edit View Search Tools Documents Help
Open Save Undo
Nuevo Documento de texto.txt *procesar_ventas.pig
ventas = LOAD '/user/cloudera/blq2-tarea1/compras.csv' USING PigStorage
('\'073\') AS (id:int, id_cliente:int, producto:chararray, precio:float);
ventas_id_y_precio = FOREACH ventas GENERATE id_cliente, precio;
ventas_id_y_precio_limpio = FILTER ventas_id_y_precio BY (precio is not null);
STORE ventas_id_y_precio_limpio INTO '/user/cloudera/blq2-tarea1/tabla_ventas'
USING PigStorage(',');
```

Script 3: Hive (consulta final)

gedit ~/blq2-tarea1/accion_oozie/bin/consulta_resultado.hive

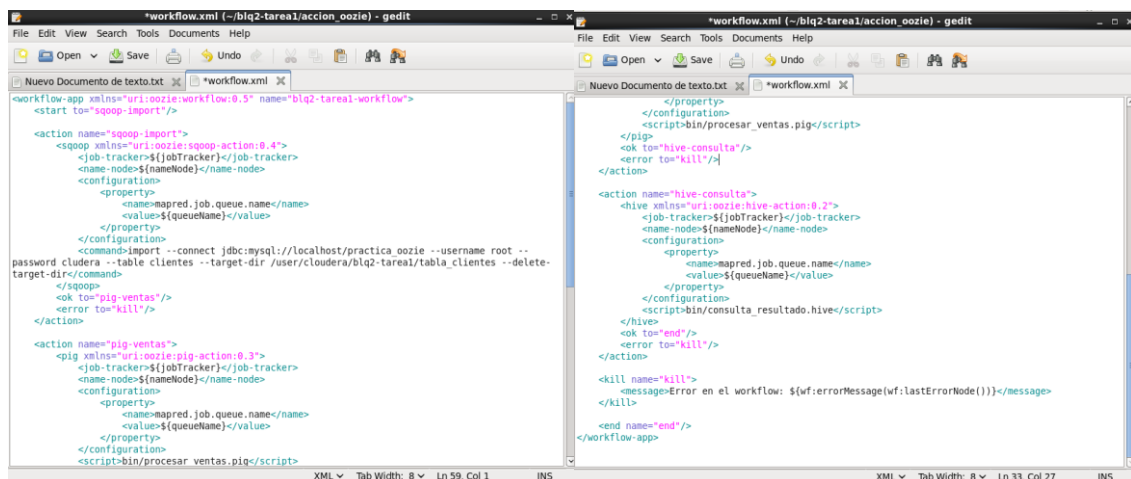


```
*consulta_resultado.hive (~/blq2-tarea1/accion_oozie/bin) - gedit
File Edit View Search Tools Documents Help
Open Save Undo
Nuevo Documento de texto.txt *consulta_resultado.hive
CREATE DATABASE IF NOT EXISTS blq2tarea1;
USE blq2tarea1;
DROP TABLE IF EXISTS ventas;
CREATE EXTERNAL TABLE ventas (
  id INT,
  precio FLOAT
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/cloudera/blq2-tarea1/tabla_ventas';
DROP TABLE IF EXISTS clientes;
CREATE EXTERNAL TABLE clientes (
  id INT,
  nombre STRING,
  email STRING,
  genero STRING,
  ip STRING
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/cloudera/blq2-tarea1/tabla_clientes';
INSERT OVERWRITE DIRECTORY '/user/cloudera/blq2-tarea1/resultado'
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073'
SELECT SUM(v.precio) AS precio_sumado, c.nombre, c.email
FROM ventas v
JOIN clientes c ON (v.id = c.id)
GROUP BY c.id, c.nombre, c.email
ORDER BY precio_sumado DESC
LIMIT 1;
```

5.3: Crear el workflow.xml (el orquestador de Oozie)

Es el archivo que conectará Sqoop, Pig y Hive de forma lineal.

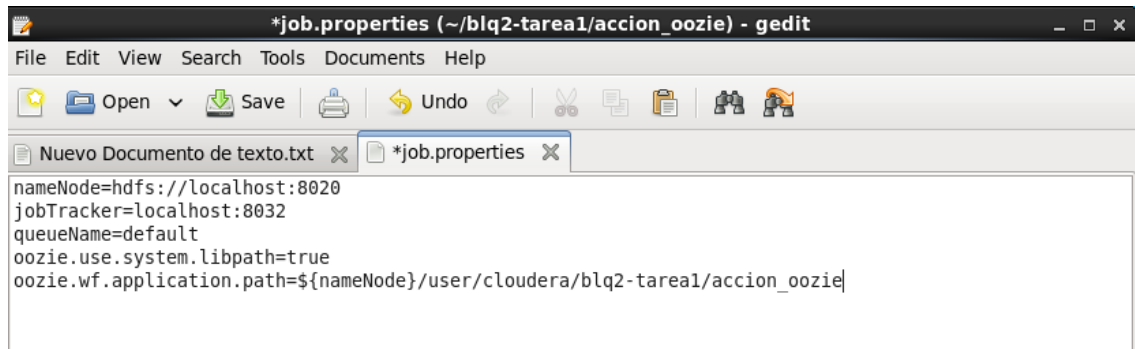
gedit ~/blq2-tarea1/accion_oozie/workflow.xml



```
*workflow.xml (~/blq2-tarea1/accion_oozie) - gedit
File Edit View Search Tools Documents Help
Open Save Undo
Nuevo Documento de texto.txt *workflow.xml
<?xml version="1.0" encoding="UTF-8"?>
<workflow-app xmlns="uri:oozie:workflow:0.5" name="blq2-tarea1-workflow">
  <start to="sqoop-import"/>
  <action name="sqoop-import">
    <sqoop xmlns="uri:oozie:sqoop-action:0.4">
      <job-tracker>${jobTracker}</job-tracker>
      <name-node>${nameNode}</name-node>
      <configuration>
        <property>
          <name>mapred.job.queue.name</name>
          <value>${queueName}</value>
        </property>
      </configuration>
      <command>import --connect jdbc:mysql://localhost/practica_oozie --username root --password cludera --table clientes --target-dir /user/cloudera/blq2-tarea1/tabla_clientes --delete-target-dir</command>
    </sqoop>
    <ok to="pig-ventas"/>
    <error to="kill"/>
  </action>
  <action name="pig-ventas">
    <pig xmlns="uri:oozie:pig-action:0.3">
      <job-tracker>${jobTracker}</job-tracker>
      <name-node>${nameNode}</name-node>
      <configuration>
        <property>
          <name>mapred.job.queue.name</name>
          <value>${queueName}</value>
        </property>
      </configuration>
      <script-bin/procesar_ventas.pig</script>
    </pig>
    <ok to="hive-consulta"/>
    <error to="kill"/>
  </action>
  <action name="hive-consulta">
    <hive xmlns="uri:oozie:hive-action:0.2">
      <job-tracker>${jobTracker}</job-tracker>
      <name-node>${nameNode}</name-node>
      <configuration>
        <property>
          <name>mapred.job.queue.name</name>
          <value>${queueName}</value>
        </property>
      </configuration>
      <script-bin/consulta_resultado.hive</script>
    </hive>
    <ok to="end"/>
    <error to="kill"/>
  </action>
  <kill name="kill">
    <message>Error en el workflow: ${wf:errorMessage(wf:lastErrorNode())}</message>
  </kill>
</workflow-app>
```

5.4: Crear job.properties (configuración)

gedit ~/blq2-tarea1/accion_oozie/job.properties



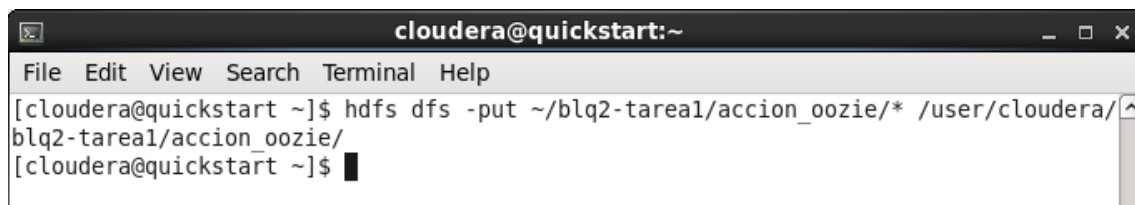
The screenshot shows a gedit window titled '*job.properties (~/blq2-tarea1/accion_oozie) - gedit'. The window contains the following text:

```
nameNode=hdfs://localhost:8020
jobTracker=localhost:8032
queueName=default
oozie.use.system.libpath=true
oozie.wf.application.path=${nameNode}/user/cloudera/blq2-tarea1/accion_oozie/
```

5.5: Subir todo a HDFS

Copiamos todos los archivos desde LOCAL al sistema HDFS:

hdfs dfs -put ~/blq2-tarea1/accion_oozie/ /user/cloudera/blq2-tarea1/accion_oozie/*

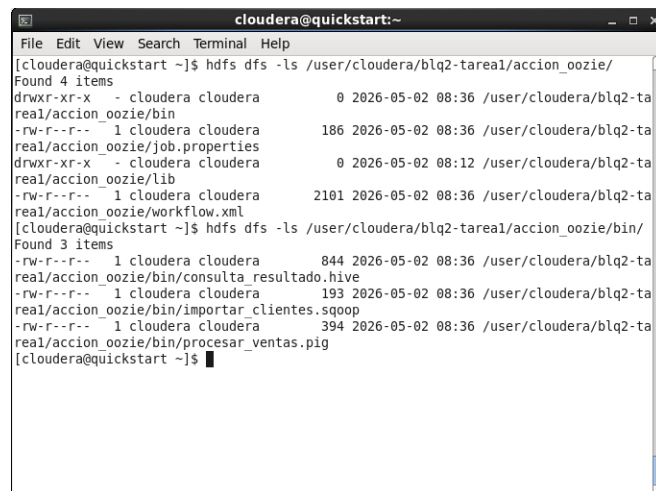


The screenshot shows a terminal window titled 'cloudera@quickstart:~'. The command `hdfs dfs -put ~/blq2-tarea1/accion_oozie/* /user/cloudera/blq2-tarea1/accion_oozie/` has been executed, and the prompt is now `[cloudera@quickstart ~]$`.

Verificamos

hdfs dfs -ls /user/cloudera/blq2-tarea1/accion_oozie/

hdfs dfs -ls /user/cloudera/blq2-tarea1/accion_oozie/bin/



The screenshot shows a terminal window titled 'cloudera@quickstart:~'. The command `hdfs dfs -ls /user/cloudera/blq2-tarea1/accion_oozie/` has been executed, showing 4 items:

```
drwxr-xr-x - cloudera cloudera 0 2026-05-02 08:36 /user/cloudera/blq2-tarea1/accion_oozie/bin
-rw-r--r-- 1 cloudera cloudera 186 2026-05-02 08:36 /user/cloudera/blq2-tarea1/accion_oozie/job.properties
drwxr-xr-x - cloudera cloudera 0 2026-05-02 08:12 /user/cloudera/blq2-tarea1/accion_oozie/lib
-rw-r--r-- 1 cloudera cloudera 2101 2026-05-02 08:36 /user/cloudera/blq2-tarea1/accion_oozie/workflow.xml
```

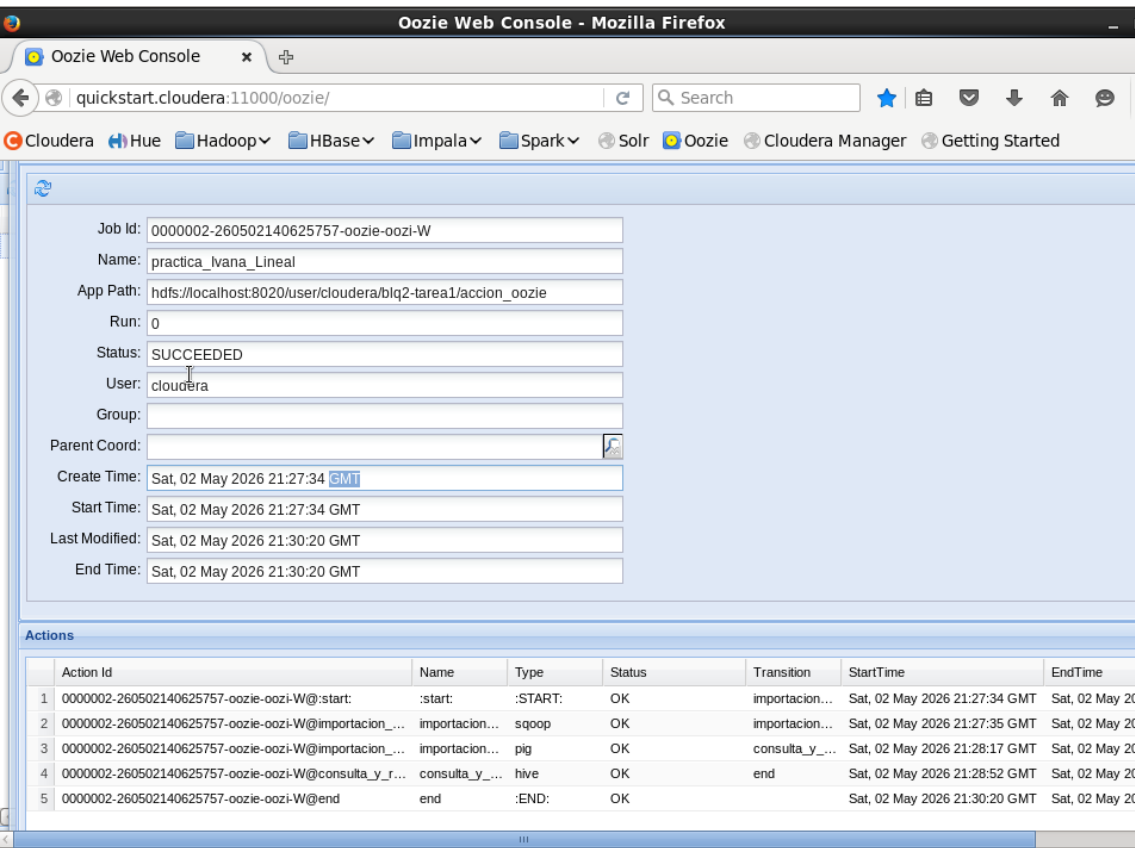
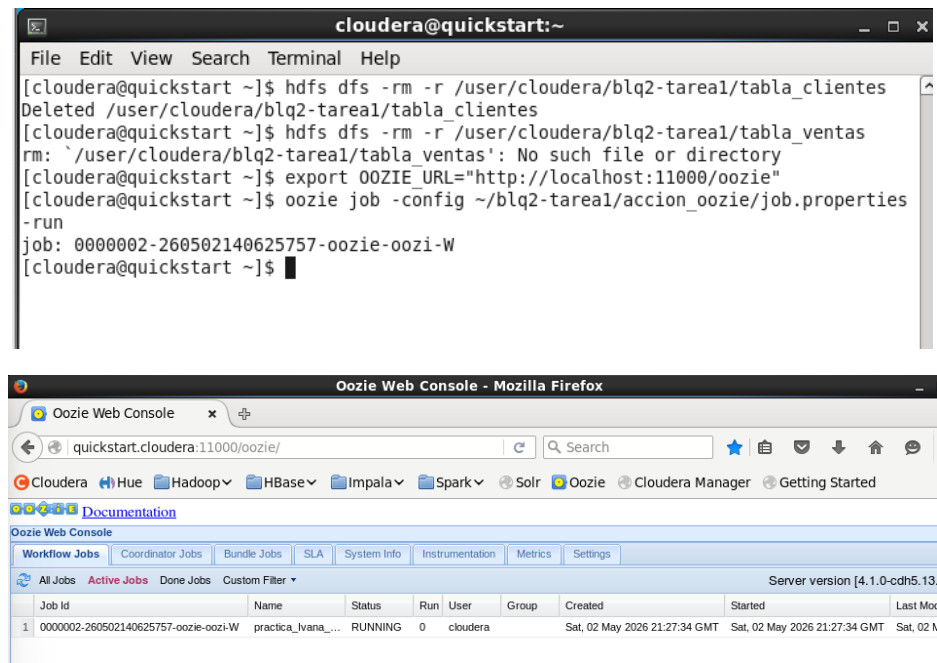
The command `hdfs dfs -ls /user/cloudera/blq2-tarea1/accion_oozie/bin/` has been executed, showing 3 items:

```
-rw-r--r-- 1 cloudera cloudera 844 2026-05-02 08:36 /user/cloudera/blq2-tarea1/accion_oozie/bin/consulta_resultado.hive
-rw-r--r-- 1 cloudera cloudera 193 2026-05-02 08:36 /user/cloudera/blq2-tarea1/accion_oozie/bin/importar_clientes.sqoop
-rw-r--r-- 1 cloudera cloudera 394 2026-05-02 08:36 /user/cloudera/blq2-tarea1/accion_oozie/bin/procesar_ventas.pig
```

5.6: Lanzar Oozie

```
export OOZIE_URL="http://localhost:11000/oozie"
```

```
oozie job -config ~/blq2-tarea1/accion_oozie/job.properties -run
```



El job finaliza en estado SUCCEEDED, validando la correcta integración de todas las herramientas.

Desarrollo del Pipeline

Paso 1: Preparación de Orígenes

- **HDFS:** Se creó el directorio blq2-tarea1 y se cargó compras.csv desde la carpeta compartida /media/sf_datos/.
- **MySQL:** Se creó la base de datos practica_oozie y se pobló la tabla clientes mediante el script clientes.sql.

Paso 2: Ingesta con Sqoop

Se configuró la acción para importar la tabla de clientes. Se utilizó el parámetro --delete-target-dir para permitir la re-ejecución del workflow sin errores manuales de borrado.

- **Resultado:** Datos almacenados en /user/cloudera/blq2-tarea1/tabla_clientes.

Paso 3: Transformación con Pig

El script importacion_ventas.pig procesó el CSV original.

- **Limpieza:** Se eliminaron filas con precios nulos.
- **Formateo:** Para evitar conflictos de lectura en Hive, se almacenó el resultado final utilizando el separador **punto y coma (;)**.

Paso 4: Análisis y Join con Hive

Para la acción de Hive, se utilizó la **versión optimizada para Oozie** basada en tablas externas.

- **Configuración:** Se definieron los delimitadores específicos para cada tabla (',' para clientes y ';' para ventas) para evitar valores nulos en el cruce de datos.
- **Consulta:** Se realizó un JOIN por id_cliente, agrupando por usuario y sumando sus compras para identificar al mayor consumidor.

Resultados y Conclusiones

1. Validación en Oozie

Como se muestra en las capturas de la Oozie Web Console, el flujo lineal se completó con éxito. El orquestador coordinó correctamente el paso de datos entre Sqoop, Pig y Hive, terminando todas las acciones en estado OK.

2. Respuesta a la pregunta: ¿Quién ha gastado más dinero?

Tras la ejecución, el archivo de salida en HDFS arrojó el siguiente resultado:

- **Total gastado:** 416.36
- **Nombre del cliente:** Car
- **Email:** cboycott1z@dion.ne.jp

3. Conclusión Técnica

La práctica demuestra la potencia de Oozie para integrar herramientas heterogéneas. La resolución de errores críticos (como el ajuste de separadores en Hive y la simplificación del flujo por falta de recursos) fue clave para transformar un pipeline propenso a fallos en una solución robusta y funcional en un entorno de recursos limitados.

OPCIÓN 2: AUTOMATIZACIÓN CON APACHE AIRFLOW Y PYSPARK

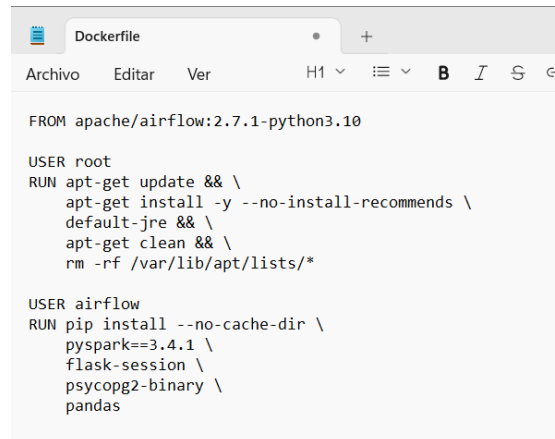
Introducción y selección del entorno

Para esta opción utilizaremos nuestra máquina real, pues Cloudera no posee la infraestructura necesaria para ejecutar versiones modernas de orquestadores.

Hemos garantizado un entorno estable y profesional, utilizando la imagen oficial `apache/airflow:2.7.1-python3.10`. La elección de Python 3.10 sobre versiones más recientes se debe a la necesidad de compatibilidad total con las librerías de PySpark y los conectores de base de datos.

Configuración del dockerfile

Se ha diseñado un Dockerfile personalizado para incluir las dependencias de sistema necesarias para Spark (Java) y las librerías de análisis de datos.



```
FROM apache/airflow:2.7.1-python3.10

USER root
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
    default-jre && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

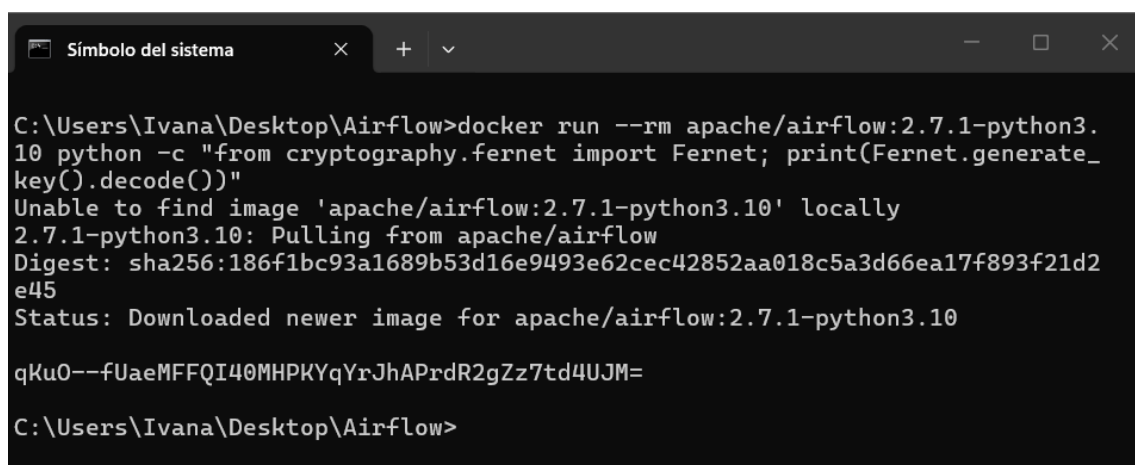
USER airflow
RUN pip install --no-cache-dir \
    pyspark==3.4.1 \
    flask-session \
    psycogp2-binary \
    pandas
```

Preparación de Seguridad y Base de Datos

Airflow requiere una clave de cifrado (Fernet Key) y una base de datos inicializada para operar. Se siguieron estos pasos críticos en la terminal:

- **Generación de Llave de Seguridad:** Se ejecutó un contenedor temporal para generar una clave válida.

```
docker run --rm apache/airflow:2.7.1-python3.10 python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```



```
Símbolo del sistema
C:\Users\Ivana\Desktop\Airflow>docker run --rm apache/airflow:2.7.1-python3.10 python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
Unable to find image 'apache/airflow:2.7.1-python3.10' locally
2.7.1-python3.10: Pulling from apache/airflow
Digest: sha256:186f1bc93a1689b53d16e9493e62cec42852aa018c5a3d66ea17f893f21d2e45
Status: Downloaded newer image for apache/airflow:2.7.1-python3.10
qKu0--fUaeMFFQI40MHPKYqYrJhAPrdR2gZz7td4UJM=
C:\Users\Ivana\Desktop\Airflow>
```

- **Inicialización de Metadatos:** Se migraron las tablas del sistema a la base de datos PostgreSQL interna.

docker-compose run --rm airflow-webserver airflow db migrate

```
Símbolo del sistema
C:\Users\Ivana\Desktop\Airflow>docker-compose run --rm airflow-webserver airflow db migrate
time="2026-05-03T14:33:24+02:00" level=warning msg="C:\\Users\\Ivana\\Desktop\\Airflow\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] 3/3t 3/33
  ✓ Network airflow_default          Created          0.0s
  ✓ Volume airflow_postgres_data     Created          0.0s
  ✓ Container airflow-postgres-1     Started          0.4s
Container airflow-airflow-webserver-run-274afb3d375f Creating
Container airflow-airflow-webserver-run-274afb3d375f Created

DB: postgresql+psycopg2://airflow:***@postgres:5432/airflow
Performing upgrade to the metadata database postgresql+psycopg2://airflow:***@postgres:5432/airflow
[2026-05-03T12:33:31.546+0000] {migration.py:213} INFO - Context impl PostgresqlImpl.
[2026-05-03T12:33:31.546+0000] {migration.py:216} INFO - Will assume transactional DDL.
[2026-05-03T12:33:31.548+0000] {migration.py:213} INFO - Context impl PostgresqlImpl.
[2026-05-03T12:33:31.548+0000] {migration.py:216} INFO - Will assume transactional DDL.
INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO [alembic.runtime.migration] Will assume transactional DDL.
INFO [alembic.runtime.migration] Running stamp_revision -> 405de8318b3a
Database migrating done!

C:\Users\Ivana\Desktop\Airflow>
```

- **Creación del Administrador:** Se dio de alta al usuario principal para acceder a la interfaz web.

docker-compose run --rm airflow-webserver airflow users create --username airflow --firstname Ivana --lastname Practica --role Admin --email admin@example.com --password airflow

```
C:\Users\Ivana\Desktop\Airflow>docker-compose run --rm airflow-webserver airflow users create --username airflow --firstname Ivana --lastname Practica --role Admin --email admin@example.com --password airflow
time="2026-05-03T14:36:25+02:00" level=warning msg="C:\\Users\\Ivana\\Desktop\\Airflow\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] 1/1t 1/11
  ✓ Container airflow-postgres-1     Running          0.0s
Container airflow-airflow-webserver-run-881cecbd6efd Creating
Container airflow-airflow-webserver-run-881cecbd6efd Created

/home/airflow/.local/lib/python3.10/site-packages/flask_limiter/extension.py
```


Ahora ya podemos ejecutar el archivo de Docker Compose desde su ubicación.

```
Símbolo del sistema - docker
C:\Users\Ivana\Desktop\Airflow>docker-compose up -d --build
time="2026-05-03T14:12:59+02:00" level=warning msg="C:\\Users\\Ivana\\Deskt
p\\Airflow\\docker-compose.yml: the attribute 'version' is obsolete, it will
be ignored, please remove it to avoid potential confusion"
[+] up 14/17
- Image postgres:13 [###.....] 41.67MB / 161.4MB Pulling 9.3s
```

```
#10 [airflow-webserver] resolving provenance for metadata file
#10 DONE 0.1s

[+] up 24/24-scheduler] resolving provenance for metadata file
✓Image postgres:13 Pulled 33.9s
✓Image airflow-airflow-scheduler Built 174.8s
✓Image airflow-airflow-webserver Built 174.8s
✓Network airflow_default Created 0.1s
✓Volume airflow_postgres_data Created 0.0s
✓Container airflow-postgres-1 Created 0.3s
✓Container airflow-airflow-webserver-1 Created 0.1s
✓Container airflow-airflow-scheduler-1 Created 0.1s

C:\Users\Ivana\Desktop\Airflow>
```

```
Símbolo del sistema
C:\Users\Ivana\Desktop\Airflow>docker-compose up -d
time="2026-05-03T14:37:37+02:00" level=warning msg="C:\\Users\\Ivana\\Deskt
p\\Airflow\\docker-compose.yml: the attribute 'version' is obsolete, it will
be ignored, please remove it to avoid potential confusion"
[+] up 3/3
✓Container airflow-postgres-1 Running 0.0s
✓Container airflow-airflow-webserver-1 Created 0.4s
✓Container airflow-airflow-scheduler-1 Created 0.4s

C:\Users\Ivana\Desktop\Airflow>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                                          NAMES
c83d402603d1   airflow-airflow-webserver           "/usr/bin/dumb-init ..." 7 second
s ago        Up 6 seconds  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp airflow
-airflow-webserver-1
85c164e48f84   airflow-airflow-scheduler           "/usr/bin/dumb-init ..." 7 second
s ago        Up 6 seconds  8080/tcp                airflow
-airflow-scheduler-1
9d3253a43094   postgres:13                         "docker-entrypoint.s..." 4 minute
s ago        Up 4 minutes  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp airflow
-postgres-1

C:\Users\Ivana\Desktop\Airflow>
```

Implementación del DAG (Pipeline de Ventas)

Se ha implementado un flujo de trabajo bajo el nombre `pipeline_diario_ventas_pyspark`. Este pipeline utiliza el motor de Spark para realizar las siguientes tareas automatizadas:

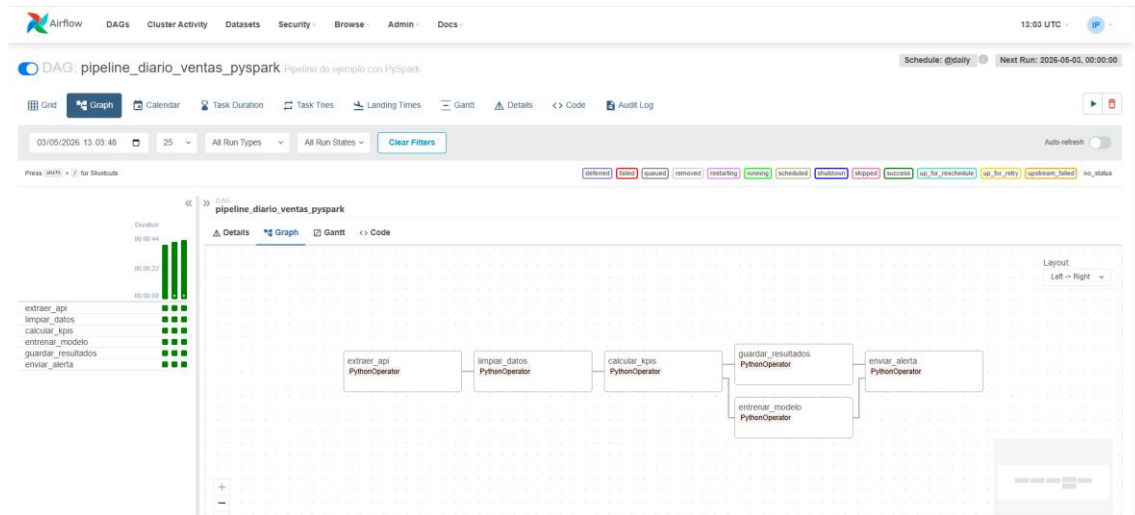
1. **extraer_api**: Simulación de ingesta de datos.
2. **limpiar_datos**: Procesamiento de nulos mediante DataFrames de Spark.
3. **calcular_kpis**: Agregación de ventas y cálculos financieros.
4. **Ejecución en Paralelo**: El flujo realiza simultáneamente el entrenamiento de un modelo (`entrenar_modelo`) y el guardado de resultados (`guardar_resultados`), optimizando el tiempo total de ejecución.

En el navegador vamos a `localhost:8080` y activamos el DAG (interruptor de la izquierda) y posteriormente seleccionamos **Trigger DAG** para ejecutar nuestra tarea.

The screenshot displays the Apache Airflow web interface. The top section shows the DAGs list with a table containing columns for DAG, Owner, Runs, Schedule, Last Run, Next Run, Recent Tasks, Actions, and Links. The DAG 'pipeline_diario_ventas_pyspark' is highlighted, and a red arrow points to the 'Trigger DAG' button in the Actions column. Below this, the task details for 'calcular_kpis' are shown, including a Gantt chart, task instance notes, and a table of task instance details.

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions	Links
pipeline_diario_ventas_pyspark	airflow	1	@daily	2026-05-02, 00:00:00	2026-05-02, 00:00:00	extraer_api, limpiar_datos, calcular_kpis, entrenar_modelo, guardar_resultados, enviar_alerta	Trigger DAG	

Task	Status	Task ID	Run ID	Operator	Trigger Rule	Duration	Started	Ended
calcular_kpis	success	calcular_kpis	manual_2026-05-03T13:01:53.507416+00:00	PythonOperator	all_success	00:00:06	2026-05-03, 13:02:21 UTC	2026-05-03, 13:02:28 UTC



Resultados y Conclusiones

Como se observa en la captura de la interfaz de Airflow, el pipeline se ha ejecutado con éxito (estado SUCCEEDED). Todos los nodos del grafo se muestran en verde oscuro, lo que confirma que:

- El entorno de Docker ha gestionado correctamente los recursos entre los contenedores de Postgres, el Scheduler y el Webserver.
- Las consultas de PySpark se han ejecutado satisfactoriamente sobre el clúster local.
- La automatización elimina la necesidad de intervenciones manuales, quedando el proceso listo para su ejecución programada diaria.